# Code Walkthrough – Enterprise PSA (SAP CAP + HANA + Fiori + XSUAA)

This guide explains every file of the project, in the order you would create it by hand, and what each line or block does. The folder layout follows the sample project SEP-22-2026-SKILLS-CAPJS (approuter in `app/`, one `annotations.cds` per app, `db/schema.cds` + `db/CDSView.cds` + `db/i18n`, `srv/service.cds` + `srv/service.js`). Follow it top to bottom in SAP Business Application Studio (BAS) to rebuild the project from an empty folder.

---

## 0. The big picture

```
Browser ──▶ Approuter (app/) ──▶ CAP server (srv/) ──▶ SAP HANA Cloud (db/)
            |  XSUAA login, /do/logout          |  OData V4 /odata/v4/psa
            |  serves app/index.html and        |  roles, validations, roll-up, invoices
            |  app/<name>/webapp itself,        |
            |  forwards /odata with the token   |
            ▼
   SAP Fiori launchpad (app/index.html) with 9 Fiori Elements apps (app/<name>/webapp)
   – the screens are generated from app/<name>/annotations.cds (collected by
app/services.cds)
```

| Layer | Folder / file | Technology |
|---|---|---|
| Data model | `db/common.cds`, `db/schema.cds`, `db/CDSView.cds`, `db/i18n/*`, `db/data/*.csv` | CDS → SAP HANA Cloud tables and views |
| Service | `srv/service.cds`, `srv/test.http` | CAP OData V4 service, authorization |
| Business logic | `srv/service.js` | Node.js event handlers |
| UI | `app/services.cds`, `app/<app>/annotations.cds`, `app/<app>/webapp/*`, `app/index.html` | Fiori Elements, SAPUI5, Fiori launchpad sandbox |
| Security | `xs-security.json`, `app/package.json`, `app/xs-app.json` | XSUAA, SAP Approuter |
| Deployment | `mta.yaml`, `.pipeline/config.yml` | MTA on Cloud Foundry, SAP CI/CD |
| Tests | `test/psa.test.js` | `node --test` + `@cap-js/cds-test` |

**Request flow (example: Admin approves a timesheet)**

1. The Fiori app calls `POST /odata/v4/psa/ApprovalSrv(<key>)/approveTimesheet` through the approuter.
2. The approuter checks the XSUAA session and forwards the call with the user's token to CAP.
3. CAP checks `@restrict` (Admin only), then runs `this.on('approveTimesheet', …)` in `service.js`.
4. The handler sets the status, then `recalculateProject()` updates task → milestone → project → budget → alerts in HANA.
5. Fiori refreshes the page (because of `Common.SideEffects`) and shows the toast message.

---

## 1. Create the project (BAS)

1. BAS → create a Dev Space **Full Stack Cloud Application**, open it, open a terminal.
2. Run the following commands:

```
cd ~/projects
cds init psa-application                # creates package.json, db/, srv/, app/ and .cdsrc
cd psa-application
npm i @sap/cds @cap-js/hana @sap/xssec   # runtime: CAP, HANA driver, XSUAA token check
npm i -D @sap/cds-dk @cap-js/sqlite @cap-js/cds-test   # dev: CLI, local DB for tests, test
helper
npm i -D mbt rimraf                     # dev: MTA build tool, clean-up of old archives
```

3. Create these folders: `db/data`, `db/i18n`, `test`, `.pipeline`, `docs` (`app/` and `srv/` exist already; the approuter goes directly into `app/`, chapter 12.2).
4. Create the files in the order of the following chapters.

## 2. `package.json` – project definition

```
{
  "name": "psa-application",                 // npm package name, also used by mta.yaml
  "version": "1.0.0",
  "description": "Professional Services Automation (PSA) - SAP CAP Node.js",
  "dependencies": {
    "@cap-js/hana": "^3",                    // SAP HANA database adapter (production)
    "@sap/cds": "^10",                       // CAP runtime
    "@sap/xssec": "^4"                       // validates XSUAA JWT tokens
  },
  "devDependencies": {
    "@cap-js/cds-test": "^1",                // GET/POST helpers for tests
    "@cap-js/sqlite": "^3",                  // in-memory DB for local runs and tests
    "@sap/cds-dk": "^10",                    // cds CLI: cds watch, cds build, cds deploy
    "mbt": "^1.2.49",                        // Cloud MTA Build Tool (npm run build)
    "rimraf": "^5.0.5"                       // deletes old build results
  },
  "scripts": {
    "start": "cds-serve",                    // how Cloud Foundry starts the server
    "watch": "cds watch --open index.html",   // local development with auto-restart, opens
the launchpad
    "watch-projects": "cds watch --open projects/webapp/index.html?sap-ui-xx-viewCache=false
--livereload false",
                                             // one script per app: watch-home, watch-
timesheets, … watch-auditlog
    "test": "node --test test/psa.test.js",   // run the tests (development profile: SQLite +
mocked users)
    "build": "rimraf resources mta_archives && mbt build --mtar archive",   // ->
mta_archives/archive.mtar
    "deploy": "cf deploy mta_archives/archive.mtar --retries 1",
    "undeploy": "cf undeploy psa-application --delete-services --delete-service-keys --
delete-service-brokers"
  },
  "sapux": [ "app/home", "app/projects", … "app/auditlog" ],   // tells SAP Fiori tools in
BAS where the apps are
  "cds": {
    "requires": {
      "[development]": {                     // local runs (cds watch) and npm test
        "auth": {                            // the database is SQLite in memory by default
          "kind": "mocked",                  // fake logins (basic auth)
          "users": {
            "meera.iyer@psa-demo.com":   { "password": "admin", "roles": ["Admin"] },
            "rahul.sharma@psa-demo.com": { "password": "user",  "roles": ["User"] },
            "alice": false, "bob": false, …, "*": false   // switch off CAP's default demo
users
          }
        }
      },
      "[hybrid]":     { "db": "hana", "auth": "xsuaa" },   // cds watch --profile hybrid (BAS
+ real BTP services)
      "[production]": { "db": "hana", "auth": "xsuaa" }    // Cloud Foundry
    }
  },
  "engines": { "node": ">=22" }              // Node version for Cloud Foundry / BAS
}
```

The code uses CommonJS (`require` / `module.exports`), like the sample project, so there is no `"type": "module"`.

*(Comments are only for explanation – JSON itself does not allow comments.)*

---

## 3. `db/common.cds` – reusable types

```
namespace psa.common;                          // all names below start with psa.common.
using { Currency } from '@sap/cds/common';     // SAP standard currency code list
(sap.common.Currencies)
```

**Simple types** – one place to change the length/precision of many fields:

```
type Guid        : UUID;            // all keys (NODE_KEY) are UUIDs
type PhoneNumber : String(32);
type Email       : String(255);
type String32    : String(32);
type String64    : String(64);
type String255   : String(255);
type AmountT : Decimal(15,2);       // money: 13 digits + 2 decimals
type Hours   : Decimal(10,2);
type Percent : Decimal(7,2);
```

**Enums** – a String type with a fixed list of allowed values. With `@assert.range` (in the service) CAP rejects any other value. Fiori shows them as normal text fields.

```
type BillingType : String(10) enum { fixed = 'Fixed'; tm = 'T&M'; retainer = 'Retainer'; };
type ProjectStatus   : … 'Draft' | 'Active' | 'On Hold' | 'Completed' | 'Cancelled'
type Priority        : … 'Low' | 'Medium' | 'High' | 'Critical'
type MilestoneStatus : … 'Not Started' | 'In Progress' | 'Completed' | 'Delayed'
type ClientStatus    : … 'Active' | 'Inactive' | 'Prospect'
type BudgetStatus    : … 'Within Budget' | 'At Risk' | 'Breached'
type TaskStatus      : … 'To Do' | 'In Progress' | 'Done' | 'Blocked'
type TimesheetStatus : … 'Draft' | 'Submitted' | 'Approved' | 'Rejected' | 'Invoiced'
type InvoiceStatus   : … 'Draft' | 'Issued' | 'Paid' | 'Cancelled'   // "Overdue" is
calculated
```

**Aspects** – a group of fields that is "mixed into" entities with `entity X : Amount { … }`:

```
aspect Amount  { GROSS_AMOUNT : AmountT @(title: '{i18n>GROSS_AMOUNT}'); NET_AMOUNT …;
TAX_AMOUNT …;
                 CURRENCY : Currency @(title: '{i18n>CURRENCY_CODE}'); }
                                        // CURRENCY becomes a foreign key column
CURRENCY_code
aspect Address { STREET @(title: '{i18n>STREET}'); POSTAL_CODE; CITY; COUNTRY; BUILDING; }
```

`@(title: '{i18n>KEY}')` = the field label, read from `db/i18n/i18n.properties` (chapter 5.3).

---

## 4. `db/schema.cds` – part 1: `context master` (master data)

`db/schema.cds` holds all tables in two contexts, like the sample project: `master` (this chapter) and `transaction` (chapter 5).

```
namespace psa.db;                                   // tables are named psa_db_master_Clients
…
using { psa.common as cmn } from './common';
using { cuid, managed } from '@sap/cds/common';     // cuid = key ID : UUID; managed =
createdAt/By, modifiedAt/By
using { CDSViews } from './CDSView';                // the view ProjectTimeEntries (chapter
5.2)

context master {                                    // a context groups entities:
psa.db.master.Clients
```

Every field gets a label from the i18n file, e.g. `CLIENT_NAME : cmn.String255 @(title: '{i18n>CLIENT_NAME}');` (left out in the short listings below).

**Clients**

```
    entity Clients : cmn.Address, managed {         // + 5 address fields + 4 managed fields
        key NODE_KEY : cmn.Guid  @(title: '{i18n>NODE_KEY}');   // primary key (sample-
project style)
        CLIENT_NAME    : cmn.String255;
        CONTACT_PERSON : cmn.String64;
        EMAIL          : cmn.Email;
        PHONE          : cmn.PhoneNumber;
        INDUSTRY       : cmn.String64;
        STATUS         : cmn.ClientStatus default 'Active';   // default used on insert
    }
```

**Employees** – also the login link: `email` must equal the XSUAA login e-mail.

```
    entity Employees : cuid, managed {              // key ID : UUID (from cuid)
        nameFirst; nameLast : cmn.String64;
        email : cmn.Email;  phoneNumber; jobTitle; joinDate : Date;
        Currency_code : String(3);  hourlyRate : cmn.AmountT;   // internal cost rate
        isManager : Boolean default false;          // "Manager" of the FRD = employee with
this flag
        isActive  : Boolean default true;           // inactive employees cannot log time
        manager   : Association to Employees;       // line manager -> column manager_ID
    }
```

**Settings** – one row (ID = 1) the Admin can edit; read by the business logic.

```
    entity Settings : managed {
        key ID : Integer;
        ALERT_THRESHOLDS : String(50) default '75,90,100';   // comma separated
        AT_RISK_PERCENT  : cmn.Percent default 80;
        TAX_PERCENT      : cmn.Percent default 18;
        PAYMENT_DAYS     : Integer default 30;
    }
}
```

## 5. `db/schema.cds` – part 2: `context transaction`, `db/CDSView.cds`, `db/i18n`

### 5.1 `context transaction` (same file, below `context master`)

```
context transaction {                                  // master.Clients / master.Employees can
be used directly
```

**Key ideas**

- `Association to X` = a foreign key (`X_NODE_KEY` column). Used for links (project → client).
- `Composition of …` = parent/child; children are created, edited and deleted **with** the parent. Fiori drafts edit a project together with its budget, milestones and tasks because of this.
- `on CHILD.PARENT = $self` = the "backlink": the child has the foreign key to the parent.

**Projects**

```
    entity Projects : managed {
        key NODE_KEY : cmn.Guid;
        PROJECT_CODE; PROJECT_NAME; DESCRIPTION : String(500);
        BILLING_TYPE : cmn.BillingType;
        STATUS   : cmn.ProjectStatus default 'Active';
        PRIORITY : cmn.Priority default 'Medium';
        START_DATE; END_DATE : Date;
        PLANNED_HOURS : cmn.Hours;
        CONSUMED_HOURS : cmn.Hours default 0;    // filled by the roll-up
        CLIENT  : Association to master.Clients;    // CLIENT_NODE_KEY
        MANAGER : Association to master.Employees;  // MANAGER_ID
        BUDGET     : Composition of one Budgets   on BUDGET.PROJECT = $self;      // exactly
one budget
        MILESTONES : Composition of many Milestones on MILESTONES.PROJECT = $self;
        ALERTS     : Association to many Alerts   on ALERTS.PROJECT = $self;      // read-
only lists
        INVOICES   : Association to many Invoices on INVOICES.PROJECT = $self;
        TIME_ENTRIES : Association to many CDSViews.ProjectTimeEntries on
TIME_ENTRIES.PROJECT = $self;
    }
```

**Budgets** (1 : 1 with a project): amount, billing rate, retainer fee, currency and the calculated `CONSUMED_AMOUNT`, `REMAINING_AMOUNT`, `CONSUMED_PERCENT`, `BUDGET_STATUS`; `PROJECT : Association to Projects`.

**Milestones**: name, description, dates, `COMPLETION_PERCENT`, `STATUS` (set by the roll-up), `PROJECT`, `TASKS : Composition of many Tasks on TASKS.MILESTONE = $self`.

**Tasks**: name, description, dates, priority, status, `ESTIMATED_HOURS`, calculated `ACTUAL_HOURS` and `BURN_PERCENT`, `MILESTONE`, `ASSIGNEE : Association to master.Employees`.

**Timesheets** (header: one employee, one week)

```
        WEEK_START; WEEK_END : Date;
        TOTAL_HOURS; BILLABLE_HOURS : cmn.Hours default 0;
        STATUS : cmn.TimesheetStatus default 'Draft';
        SUBMITTED_DATE; APPROVED_DATE : Date;  REJECTION_REASON : cmn.String255;
        EMPLOYEE, APPROVED_BY : Association to master.Employees;
        Entries : Composition of many TimesheetEntries on Entries.PARENT = $self;
```

**TimesheetEntries** (items: one day, one task)

```
        WORK_DATE : Date;  HOURS : Decimal(4,2);  IS_BILLABLE : Boolean default true;  NOTES;
        PARENT  : Association to Timesheets;        // header (reference-project naming)
        TASK    : Association to Tasks;
        INVOICE : Association to Invoices;          // set when billed -> never billed twice
```

**AuditLog**: `TIMESTAMP, USER_ID, ACTION, ENTITY, ENTITY_KEY, DETAILS` – written by the logic, never edited.

**Invoices** `: cmn.Amount, managed`: invoice no, period, invoice / due / payment date, billable hours, status, `PROJECT`, `Entries : Association to many TimesheetEntries on Entries.INVOICE = $self`.

**Alerts**: `THRESHOLD, CONSUMED_PERCENT, MESSAGE, PROJECT, TASK` (task empty = project alert).

### 5.2 `db/CDSView.cds` – read-only views

Views live in their own file in `context CDSViews` (sample-project style). On HANA they become SQL views.

```
using { psa.db.transaction } from './schema';

context CDSViews {
    // hours booked on a project: entry + task + employee + invoice in one row (Timesheets
facet)
    define view ![ProjectTimeEntries] as select from transaction.TimesheetEntries {
        key NODE_KEY, WORK_DATE, HOURS, IS_BILLABLE, NOTES,
        TASK.TASK_NAME as ![TASK_NAME],             // path expression = automatic LEFT JOIN
        @title: '{i18n>TIMESHEET_STATUS}'
        PARENT.STATUS  as ![TIMESHEET_STATUS],
        @title: '{i18n>EMPLOYEE}'
        PARENT.EMPLOYEE.nameFirst || ' ' || PARENT.EMPLOYEE.nameLast as ![EMPLOYEE_NAME] :
String(130),
        @title: '{i18n>INVOICE}'
        INVOICE.INVOICE_NO as ![INVOICE_NO],
        TASK.MILESTONE.PROJECT as ![PROJECT]        // exposes the association so Projects
can link back
    };
}
```

`schema.cds` and `CDSView.cds` use each other (`Projects.TIME_ENTRIES` points to the view, the view reads the tables) – CDS allows this.

### 5.3 `db/i18n/` – field labels in three languages

```
# i18n.properties (English, default)        i18n_de.properties          i18n_hi.properties
PROJECT_CODE=Project Code                   PROJECT_CODE=Projektcode    PROJECT_CODE=परियोजना
कोड
BUDGET_STATUS=Budget Status                 BUDGET_STATUS=Budgetstatus  BUDGET_STATUS=बजट स्थिति
…
```

- `@(title: '{i18n>PROJECT_CODE}')` on a field → CAP puts the text of the user's language into `$metadata` (browser language or `?sap-language=de`). Fiori shows it as label and column header.
- The service projections and all apps inherit these labels, so `app/<app>/annotations.cds` only sets a `@title` where a screen needs a different text (e.g. `PAYMENT_STATUS` → "Status").

---

## 6. `db/data/*.csv` – sample data

- File name = `<namespace>-<context>.<Entity>.csv`, e.g. `psa.db-master.Clients.csv`, `psa.db-transaction.Projects.csv`. CAP loads them automatically (SQLite) and deploys them to HANA.
- First line = column names. Associations use the foreign key column: `CLIENT_NODE_KEY`, `MANAGER_ID`, `PARENT_NODE_KEY`, `TASK_NODE_KEY`, `CURRENCY_code`.
- Keys are readable UUIDs (`40000000-0000-4000-8000-000000000001` = project 1) so tests can refer to them.
- Calculated columns (consumed hours, burn %, totals) are **not** in the CSVs – the server computes them at start-up (`cds.once('served')`, chapter 8.4).

---

## 7. `srv/service.cds` – the OData service

```
using { psa.db as db } from '../db/schema';
using { CDSViews } from '../db/CDSView';

@requires: 'authenticated-user'      // nobody without login gets anything
service PSAService {                 // URL: /odata/v4/psa
```

### 7.1 Building blocks used in this file

| Construct | Meaning |
|---|---|
| `entity XSrv as projection on db.…` | exposes a table under a new name; `*` = all columns |
| `@odata.draft.enabled` | Fiori "Edit / Save / Discard draft" handling (draft table created by CAP) |
| `@restrict: [{ grant: …, to: …, where: … }]` | authorization: which role may do which event; `where` = row filter |
| `@readonly` | no create/update/delete through the API |
| `@cds.redirection.target: false` | a second projection on the same table is not used as target of associations |
| `case when … then … end as X : Integer` | calculated column (searched CASE – works on HANA) |
| `virtual null as X : T` | field that is not in the DB; filled by the JS code in `after READ` |
| `actions { action a() returns String; }` | bound action: `POST …/XSrv(key)/a` |
| `function f(p : T) returns R;` | unbound read operation: `GET …/f(p=…)` |

### 7.2 Entities, one by one

| Entity | What / who | Important lines |
|---|---|---|
| `whoAmI()` + type `WhoAmI` | everyone | debug: login, roles, linked employee |
| `SettingsSrv` | Admin READ/UPDATE, draft | one settings row |
| `AuditLogSrv` | Admin READ, read-only | audit trail |
| `ClientSrv` | Admin `*`, draft | master data |
| `EmployeeSrv` | User READ `where: 'email = $user'` (own record), Admin `*` | `nameFirst \|\| ' ' \|\| nameLast as fullName` |

| Entity | What / who | Important lines |
|---|---|---|
| `ProjectSrv` | Admin `*`, draft | `HOURS_PERCENT` (case), `TIME_ENTRIES : redirected to ProjectTimeSrv`, virtual `BURN_RATE`, `PROJECTED_END`, action `recalculateBudget()` |
| `ProjectTimeSrv` | Admin READ | hours of a project (Timesheets tab) |
| type `BurnSummary` + `getBurnSummary(projectID)` | Admin | FRD 7.1 |
| `BudgetSrv` | Admin | `CRITICALITY`: Breached 1 (red), At Risk 2 (orange), else 3 (green) |
| `MilestoneSrv` | Admin | `STATUS_CRITICALITY` |
| `TaskSrv` | User READ (to pick a task), Admin `*` | `MILESTONE.PROJECT.PROJECT_CODE as PROJECT_CODE`, burn `CRITICALITY` |
| `TimesheetSrv` | User READ/WRITE/submit/import **`where: 'EMPLOYEE.email = $user'`**, Admin `*`, draft | `STATUS_CRITICALITY`, `IS_LOCKED`, actions `submitTimesheet()`, `importHours(rows)` |
| `TimesheetEntrySrv` | User `where: 'PARENT.EMPLOYEE.email = $user'` | items |
| `ApprovalSrv` | Admin, read-only | `where STATUS <> 'Draft'` (submitted, approved, rejected, invoiced); `STATUS_CRITICALITY`; `Entries : redirected to TimesheetEntrySrv on …`; actions approve / reject(reason) |
| `InvoiceSrv` | Admin, read-only | `PAYMENT_STATUS` (*Overdue* when `STATUS = 'Issued' and DUE_DATE < current_date`), actions issue / markInvoicePaid(paymentDate) / cancel |
| `InvoiceSrv.generateInvoice(in : many $self, projectID, periodFrom, periodTo)` | Admin | action bound to the invoice **list** (`POST …/InvoiceSrv/PSAService.generateInvoice`): Fiori shows it above the table, opens the new invoice and reloads the list |
| `ProjectAnalytics`, `TimesheetAnalytics`, `InvoiceAnalytics`, `EntryAnalytics` | Admin READ | flat views with **counter columns** (`case … then 1 else 0`) that Fiori sums for the KPI cards and the chart |
| `AlertSrv` | Admin READ | alert colour |

**Why `$user`?** With XSUAA, `$user` is the login (e-mail). `where: 'EMPLOYEE.email = $user'` makes CAP add this condition to every query, so a User only ever sees his own timesheets.

### 7.3 The `annotate … with { … @readonly; @assert.range; }` blocks at the end

- `@readonly` on calculated fields → Fiori shows them as text, the API ignores changes.
- `@assert.range` on enum fields → only the listed enum values are accepted.

---

## 8. `srv/service.js` – business logic

The file has the same name as the `.cds` file, so CAP loads it automatically as the implementation. It is a CommonJS module (`require` / `module.exports`), like the sample project.

### 8.1 Header (lines 1–38)

```
const cds = require('@sap/cds');
const { randomUUID } = require('crypto');          // new keys for alerts, invoices, audit
rows
const { INSERT, SELECT, UPDATE } = cds.ql;         // CAP query builders

const DEFAULT_SETTINGS = { ALERT_THRESHOLDS: '75,90,100', AT_RISK_PERCENT: 80, TAX_PERCENT:
18, PAYMENT_DAYS: 30 };
const COUNTED = ['Approved', 'Invoiced'];           // only these timesheet statuses count
against the budget

const round2  = (n) => Math.round(Number(n || 0) * 100) / 100;      // money/hours with 2
decimals
const today   = () => new Date().toISOString().slice(0, 10);         // 'YYYY-MM-DD'
const addDays = (date, days) => { … d.setUTCDate(d.getUTCDate() + days) … };   // date
arithmetic in UTC
const mondayOf = (date) => { const day = …getUTCDay();               // 0 = Sunday
                             return addDays(date, day === 0 ? -6 : 1 - day); };
```

### 8.2 The service implementation function

```
module.exports = cds.service.impl(async function () {
    const { EmployeeSrv, ProjectSrv, … , SettingsSrv, AuditLogSrv } = this.entities;  //
entity handles
```

Everything below registers **event handlers**:

- `this.before(event, entity, fn)` – runs before CAP's default handling (validate, set defaults, `req.reject`).
- `this.on(event, entity, fn)` – implements custom actions/functions.
- `this.after(event, entity, fn)` – runs after the default handling (recalculate, enrich results).
- Events: `READ`, `CREATE`, `UPDATE`, `DELETE`, `NEW` (new draft), `EDIT` (edit draft), or an action name.

### 8.3 `settings()` and the audit log

```
async function settings() {                          // reads Settings row 1, falls back to
defaults
    const row = await SELECT.one.from(SettingsSrv).where({ ID: 1 });
    const value = { ...DEFAULT_SETTINGS, ...(row ?? {}) };
    return {
        alertThresholds: String(value.ALERT_THRESHOLDS).split(',').map(Number).filter(n => n
> 0).sort((a,b) => a-b),
        atRiskPercent: Number(value.AT_RISK_PERCENT),
        taxRate: Number(value.TAX_PERCENT) / 100,     // 18 -> 0.18
        paymentDays: Number(value.PAYMENT_DAYS)
    };
}

async function audit(req, action, entity, key, details) {   // one AuditLog row
    await INSERT.into(AuditLogSrv).entries({ NODE_KEY: randomUUID(), TIMESTAMP: new
Date().toISOString(),
        USER_ID: req.user?.id ?? 'system', ACTION: action, ENTITY: entity, ENTITY_KEY: …,
DETAILS: … });
}

this.after(['CREATE', 'UPDATE', 'DELETE'], async (_, req) => {   // no entity = for ALL
entities
    const entity = req.target?.name?.split('.').pop();            // 'PSAService.ProjectSrv'
-> 'ProjectSrv'
    if (!entity || entity.endsWith('drafts') || entity === 'AuditLogSrv') return;   // skip
drafts & itself
    const key = req.data?.NODE_KEY ?? req.data?.ID ?? …;
    const changed = Object.keys(req.data ?? {}).filter(/* not keys, draft or managed fields
*/);
    await audit(req, req.event, entity, key, changed.length ? 'Fields: ' + changed.join(', ')
: null);
});
```

Drafts are only saved to the real table when the user presses *Save* (draft activation = CREATE/UPDATE), so the audit log gets one row per real save, not one per keystroke.

### 8.4 Budget roll-up – `recalculateProject(projectKey)`

Called after a project is saved, after an approval and by the *Recalculate Budget* button.

```
const { alertThresholds, atRiskPercent } = await settings();
const project    = await SELECT.one.from(ProjectSrv).where({ NODE_KEY: projectKey });
const budget     = await SELECT.one.from(BudgetSrv).where({ PROJECT_NODE_KEY: projectKey });
const milestones = await SELECT.from(MilestoneSrv).where({ PROJECT_NODE_KEY: projectKey });
const tasks      = await SELECT.from(TaskSrv).where({ MILESTONE_NODE_KEY: { in: [...keys,
'none'] } });
                                                     // 'none' avoids an empty IN () list
const entries = await
SELECT.from(TimesheetEntrySrv).columns('TASK_NODE_KEY','HOURS','IS_BILLABLE')
    .where`TASK.MILESTONE.PROJECT.NODE_KEY = ${projectKey} and PARENT.STATUS in ${COUNTED}`;
                                                     // tagged template: values are passed as
safe parameters
```

1. **Tasks**: `ACTUAL_HOURS` = sum of the task's approved hours; `BURN_PERCENT` = actual / estimated × 100 → `UPDATE(TaskSrv)`.

2. **Milestones**: `COMPLETION_PERCENT` = % of tasks with status Done; `STATUS` = Completed (100 %), Delayed (end date passed), In Progress (hours or a started task), else Not Started.
3. **Project**: `CONSUMED_HOURS` = sum of task actual hours.
4. **Budget**: `CONSUMED_AMOUNT` = billable hours × billing rate; `REMAINING_AMOUNT`; `CONSUMED_PERCENT`; `BUDGET_STATUS` = Breached (> 100 %), At Risk (≥ at-risk % from Settings), else Within Budget.
5. **Alerts**: for every threshold (75, 90, 100 …) create one alert for the project and one per task that crossed it, unless the same (task, threshold) alert already exists – so each crossing alerts exactly once.

Helpers next to it:

- `projectsOfTimesheet(key)` – which projects a timesheet booked on (to recalculate them after approval).
- `updateTimesheetTotal(key)` – `TOTAL_HOURS` and `BILLABLE_HOURS` of a timesheet.
- `cds.once('served', …)` – when the server has started, compute totals and roll-ups for the sample data.

### 8.5 Logged-in user

```
const tokenPayload = (req) => req.user?.authInfo?.token?.payload ??
req.user?.tokenInfo?.getPayload?.() ?? {};
                                        // the decoded XSUAA JWT (e-mail, origin, scopes);
empty in tests
async function currentEmployee(req) {    // login e-mail -> Employees row (case-insensitive)
    const login = String(tokenPayload(req).email ?? req.user.id).toLowerCase();
    return SELECT.one.from(EmployeeSrv).where`tolower(email) = ${login}`;
}
this.on('whoAmI', …)                     // returns user, name, email, origin, isAdmin =
req.user.is('Admin') …
this.before(['CREATE','UPDATE'], EmployeeSrv, req => { req.data.email = …
trim().toLowerCase(); });
```

### 8.6 Project validation

- `before('NEW', ProjectSrv.drafts)` – a new draft gets billing type T&M and an **empty budget row**, so the budget fields can be typed in at once.
- `before(['CREATE','UPDATE'], ProjectSrv)` – on *Save*: end ≥ start; project code unique (`SELECT … where PROJECT_CODE and NODE_KEY != own`); budget amount > 0; every milestone checked by `checkMilestone` and every task by `checkTask`. `req.reject(400, text)` stops the save and Fiori shows the text.
- `outside(child, parent)` – true if the child's start is before the parent's start or its end after the parent's end (FR-003).
- `after(['CREATE','UPDATE'], ProjectSrv)` → `recalculateProject`.

### 8.7 Burn rate – `burn(project)`, `getBurnSummary`, `after READ`

```
weeks    = max(1, (min(today, END_DATE) - START_DATE) / 7 days)
burnRate = consumedHours / weeks                      // approved hours per week
projectedEnd = today + (planned - consumed) / burnRate weeks   (today if already used up)
```

- `this.on('getBurnSummary')` returns budget figures + `burnRate` + `projectedEnd`.
- `this.after('READ', [ProjectSrv, ProjectSrv.drafts])` fills the virtual fields `BURN_RATE` and `PROJECTED_END` when Fiori asks for them.

- `this.on('recalculateBudget', ProjectSrv)` → `recalculateProject(req.params[0].NODE_KEY)` + `req.notify(...)` (toast).

### 8.8 Timesheet rules – `checkEntries(timesheet, entries)`

Returns an error text or `null`. For every entry: task selected; 0.25 ≤ hours ≤ 24; date inside the week; no second entry for the same task and day (`Set` of task+date); daily total ≤ 24 h; task exists and is not Done.

Handlers: | Handler | What it does | |---|---| | `before NEW TimesheetSrv.drafts` | new timesheet: employee = logged-in user, status Draft, week = this Monday | | `before NEW TimesheetEntrySrv.drafts` | new row: date = week start, 8 h, billable | | `before EDIT TimesheetSrv` | only Draft / Rejected can be edited (409 otherwise) | | `before DELETE TimesheetSrv` | only Draft / Rejected can be deleted | | `before CREATE TimesheetSrv` | user linked to an active employee; week moved to Monday; one timesheet per week; WEEK_END = Monday + 6; `checkEntries` | | `before UPDATE TimesheetSrv` | locked check; week cannot change; `checkEntries` | | `after CREATE/UPDATE TimesheetSrv` | `updateTimesheetTotal` | | `before CREATE/UPDATE/DELETE TimesheetEntrySrv` | same rules for single rows through the API; own timesheet only |

### 8.9 Actions

- **`submitTimesheet`** – Draft/Rejected only, not empty, `checkEntries`; sets Submitted + `SUBMITTED_DATE`; audit; toast.
- **`importHours(rows)`** (FR-005) – splits the pasted text into lines (`date;task;hours;billable;notes`), skips the header, finds the task by name (or `PROJECT_CODE / name`), ignores duplicates (same task + day), validates each row with `checkEntries`, inserts the good rows, updates totals and returns *"x imported, y duplicates ignored, z not imported"* (problems listed as a warning).
- **`approveTimesheet`** / **`rejectTimesheet(reason)`** – `checkApproval` (exists, status Submitted, approver ≠ owner → 403); approve sets Approved + approver + date and calls `recalculateProject` for every project of the timesheet; reject needs a reason.
- **`generateInvoice(projectID, periodFrom, periodTo)`** – loads project + budget; selects approved, billable, not yet invoiced entries in the period; amount by billing type (T&M = hours × rate; Fixed = same but never more than budget − already billed; Retainer = flat fee, period not billed before); tax and due date from Settings; number `INV-YYYY-0001`; links the entries to the invoice; timesheets with all billable hours billed → Invoiced; audit.
- **`moveInvoice(req, from, to, extra)`** – shared status change with check of the allowed "from" status.
- **`issueInvoice`** Draft → Issued; **`markInvoicePaid(paymentDate)`** Issued → Paid (date not in the future); **`cancelInvoice`** Draft/Issued → Cancelled, unlinks the entries and sets Invoiced timesheets back to Approved.

`req.notify(text)` = success toast in Fiori; `req.warn(text)` = warning dialog; `req.reject(code, text)` = error.

### 8.10 `srv/test.http` – try the service by hand

Sample requests for the REST Client (BAS / VS Code: *Send Request* above each `###` block): `$metadata`, `whoAmI()`, `$top` / `$skip` / `$count` / `$select` / `$filter` / `$expand`, `getBurnSummary`, `$apply`, import hours, submit, approve, reject, `generateInvoice`, settings, audit log and a 403 check. Logins in the file: `Authorization: Basic meera.iyer@psa-demo.com:admin` (Admin) or the User.

---

## 9. `app/services.cds` + `app/<app>/annotations.cds` – the screens

Fiori Elements builds every page from annotations. Each app keeps its own annotations next to its `webapp` folder, and `app/services.cds` collects them, like the sample project:

```
// app/services.cds
using from './home/annotations';
using from './projects/annotations';
…
using from './auditlog/annotations';
```

Every `app/<app>/annotations.cds` starts with `using PSAService as service from '../../srv/service';` and then `annotate service.ProjectSrv with @( … );`.

## 9.1 Annotation vocabulary used

| Annotation | Effect in the UI |
|---|---|
| `@title: 'Label'` | field label / column header |
| `@UI.Hidden` | never shown |
| `@mandatory` | red star, required |
| `@UI.MultiLineText` | text area |
| `@Common.Text: X.NAME @Common.TextArrangement: #TextOnly` | show the name instead of the UUID |
| `@Common.ValueList: { CollectionPath: 'ClientSrv', Parameters: [...] }` | value-help dialog (drop-down) |
| `@Measures.ISOCurrency: CURRENCY_CODE` / `@Measures.Unit: '%'` | amount with currency / "12 %" |
| `@Communication.IsEmailAddress` | mailto link |
| `UI.HeaderInfo` | object type name, title, description, icon (`TypeImageUrl: 'sap-icon://…'`) |
| `UI.SelectionFields` | filter bar fields of the List Report |
| `UI.LineItem` | table columns; `DataFieldForAction` = toolbar button; `![@UI.Importance]: #High` = stays visible on small screens |
| `UI.PresentationVariant` | default sort order |
| `UI.DataPoint` | a value with title; `Visualization: #Progress` = progress bar; `Criticality` = colour |
| `UI.FieldGroup #X` | a group of fields (a form block) |
| `UI.HeaderFacets` | blocks in the Object Page header |
| `UI.Facets` | sections / tabs of the Object Page (`ReferenceFacet` to a FieldGroup or a child `LineItem`, `CollectionFacet` = several groups side by side) |
| `UI.Identification` | buttons in the Object Page header |
| `Criticality: X, CriticalityRepresentation: #WithIcon` | green / orange / red text with icon |
| `UI.UpdateHidden / DeleteHidden: IS_LOCKED` | hide Edit / Delete when the field is true |
| `Core.OperationAvailable: { $edmJson: … }` | enable an action only in some states |
| `Common.SideEffects: { TargetEntities: ['in'] }` | refresh the page after the action |
| `Capabilities.InsertRestrictions.Insertable: false` | hide the *Create* button |

### 9.2 What is in which file

1. **Value helps** – in the app that uses them: client, manager, assignee (projects), task (timesheets), line manager (employees), `generateInvoice` parameters (invoices), reject reason as multi-line text (approvals).
2. **Fields** – labels come from `db/i18n` (chapter 5.3); each file only adds `@mandatory`, `@UI.Hidden` for technical fields (`CRITICALITY`, keys), quick info, multi-line text, units and labels for calculated fields.
3. **`projects/annotations.cds`** – `HeaderInfo` (triangle icon); filter fields; table with budget / hours progress bars; header: project info (client, status, manager, billing type), Budget Used, Hours Used, Budget Status, Remaining Budget; sections Overview (General, Schedule with burn rate / projected completion, Budget), Milestones & Tasks, Timesheets, Scope-Creep Alerts, Invoices, Change Log. Budget, Milestone (status colour, completion bar), Task (burn bar) and `ProjectTimeSrv` (hours list) annotations.
4. **`timesheets/annotations.cds`** – header: total, billable, status; buttons *Import Hours*, *Submit for Approval*; *Hours* table (date, task, hours, billable, notes, task burn %); *Week* group with submitted / approved dates.
5. **`approvals/annotations.cds`** – list with employee, week, status, hours (filter *Status* defaults to *Submitted* via `@Common.FilterDefaultValue`); *Approve* / *Reject* above the table (select rows) **and** in the header of the opened timesheet (`UI.Identification`), enabled only for Submitted (`Core.OperationAvailable`).
6. **`invoices/annotations.cds`** (Billing Workbench) – *Generate Invoice*, *Issue*, *Mark as Paid*, *Cancel*; status column `PAYMENT_STATUS` (shows Overdue); detail with amounts and billed hours.
7. **`clients/annotations.cds`**, **`employees/annotations.cds`** – lists, contact / address / work groups.
8. **Actions** (in the file of the app that has the button) – `OperationAvailable` (Submit/Import only for an active Draft/Rejected timesheet; Issue only Draft; Paid only Issued; Cancel Draft/Issued; Recalculate only on the saved project) and `SideEffects`.
9. **`home/annotations.cds`** (Manager Home, ALP) – on `ProjectAnalytics`:
   - `Aggregation.ApplySupported` (the entity can be grouped and summed with `$apply`);
   - `Aggregation.CustomAggregate #FIELD : 'Edm.Decimal'` + `@Aggregation.default: #SUM` on each measure (needed by KPI cards and charts);
   - `UI.Chart` (bar chart budget vs. consumed per project), `UI.LineItem` (table, widths via `@HTML5.CssDefaults`), `UI.PresentationVariant` (chart + table);
   - `UI.KPI #ActiveProjects / #AtRiskBudgets` (+ `#PendingApprovals` on `TimesheetAnalytics`): `DataPoint` (value, title, colour rule), `Detail` chart;
   - `UI.DataFieldForIntentBasedNavigation` buttons to the other apps;
   - `@Common.SemanticKey: [PROJECT_CODE]` on `ProjectSrv` (in `projects/annotations.cds`), so a row opens the project directly.
10. **Billing KPIs** (in `invoices/annotations.cds`) – `EntryAnalytics` (Unbilled Hours) and `InvoiceAnalytics` (Draft Invoices, Overdue Amount) the same way.
11. **`settings/annotations.cds`** and **`auditlog/annotations.cds`**.

**How to write it by hand:** add one block per entity, check the result in the browser (`cds watch` → open the app → *Ctrl+Shift+R*), then add the next block. The BAS *Page Map* / annotation editor can generate `LineItem`, `FieldGroup` and `Facets` for you if you prefer clicking.

---

# 10. The Fiori apps – `app/<app>/webapp`

## 10.1 Generate one app (BAS)

`View → Command Palette → Fiori: Open Application Generator` → **List Report Page** → Data source *Use a Local CAP Project* → service `PSAService` → main entity (e.g. `ProjectSrv`) → module name `projects`, namespace `psa`, page layout **Flexible Column Layout**. Repeat for each app (Manager Home: **Analytical List Page**). The generator creates the app folder like in the sample project:

```
app/projects/
  annotations.cds        UI annotations (add it to app/services.cds)
  package.json           UI5 tooling for BAS (npm start / Fiori tools preview)
  ui5.yaml               UI5 tooling config: SAPUI5 from the CDN, live reload, launchpad
preview
  eslint.config.mjs      lint rules of SAP Fiori tools
  README.md              template, main entity, how to start
  webapp/manifest.json, Component.js, index.html, i18n/i18n.properties
```

Then adjust the files as described below.

## 10.2 `manifest.json` (Projects as example)

```
"sap.app": {
  "id": "psa.projects",                        // = namespace.module, used in launchpad and
index.html
  "type": "application",
  "i18n": "i18n/i18n.properties",              // texts of this app
  "title": "{{appTitle}}", "description": "{{appDescription}}",   // {{key}} = text from
webapp/i18n
  "dataSources": { "mainService": { "uri": "/odata/v4/psa/", "type": "OData",
                                    "settings": { "odataVersion": "4.0" } } },  // the CAP
service URL
  "crossNavigation": { "inbounds": { "Project-manage": {        // intent of this app in a
launchpad
      "semanticObject": "Project", "action": "manage",
      "title": "{{Project-manage.flpTitle}}", "subTitle": "{{Project-manage.flpSubtitle}}" }
} }
},
"sap.ui5": {
  "flexEnabled": true,                         // UI adaptation / personalization allowed
  "dependencies": { "minUI5Version": "1.153.0",
                    "libs": { "sap.m": {}, "sap.ui.core": {}, "sap.fe.templates": {},
"sap.f": {} } },   // sap.f = FCL
  "models": {
    "i18n":  { "type": "sap.ui.model.resource.ResourceModel", "settings": { "bundleName":
"psa.projects.i18n.i18n" } },
    "":      { "dataSource": "mainService",
              "settings": { "operationMode": "Server", "autoExpandSelect": true,
"earlyRequests": true } },
                                               // OData V4 model; autoExpandSelect = only
load shown fields
    "@i18n": { "type": "sap.ui.model.resource.ResourceModel", "uri": "i18n/i18n.properties" }
},
  "routing": {
    "config": { "routerClass": "sap.f.routing.Router",          // router of the flexible
column layout
               "flexibleColumnLayout": { "defaultTwoColumnLayoutType":
"TwoColumnsMidExpanded",
                                         "defaultThreeColumnLayoutType":
"ThreeColumnsMidExpanded" } },   // details column gets more space
    "routes": [                                 // URL pattern -> pages shown side by side
      { "pattern": ":?query:",                    "target": ["ProjectSrvList"] },
      { "pattern": "ProjectSrv({key}):?query:",    "target": ["ProjectSrvList",
"ProjectSrvObjectPage"] },
      { "pattern": "ProjectSrv({key})/MILESTONES({key2}):?query:",
        "target": ["ProjectSrvList", "ProjectSrvObjectPage", "MilestoneSrvObjectPage"] } ],
    "targets": {
      "ProjectSrvList": { "name": "sap.fe.templates.ListReport",        // List Report
template
        "controlAggregation": "beginColumnPages", "contextPattern": "",  // left column
        "options": { "settings": { "contextPath": "/ProjectSrv", "variantManagement": "Page",
          "navigation": { "ProjectSrv": { "detail": { "route": "ProjectSrvObjectPage" } } },
// row click
          "controlConfiguration": { "@com.sap.vocabularies.UI.v1.LineItem":
              { "tableSettings": { "type": "ResponsiveTable", "selectionMode": "Multi" } } }
} } },
      "ProjectSrvObjectPage":   { "name": "sap.fe.templates.ObjectPage", … "contextPath":
"/ProjectSrv",
          "controlAggregation": "midColumnPages", "contextPattern": "/ProjectSrv({key})",
```

```
// middle column
          "navigation": { "MILESTONES": { "detail": { "route": "MilestoneSrvObjectPage" } } }
},
      "MilestoneSrvObjectPage": { "name": "sap.fe.templates.ObjectPage", … "contextPath":
"/ProjectSrv/MILESTONES",
          "controlAggregation": "endColumnPages",
// right column
          "contextPattern": "/ProjectSrv({key})/MILESTONES({key2})",
          "controlConfiguration": { "TASKS/@com.sap.vocabularies.UI.v1.LineItem":
              { "tableSettings": { "creationMode": { "name": "Inline", "createAtEnd": true }
} } } } }
  },
  "rootView": { "viewName": "sap.fe.templates.RootContainer.view.Fcl", "type": "XML",
"async": true }
                                                  // flexible column layout: list and details
side by side
}
```

`webapp/i18n/i18n.properties` has `appTitle`, `appDescription` and the launchpad title/subtitle of the inbound.

Differences in the other apps:

- **timesheets** – `Entries` table with inline creation (new rows directly in the table).
- **approvals**, **timesheets** – `selectionMode: Multi` (bulk Approve / Submit).
- **invoices** – `keyPerformanceIndicators` → KPI cards from `EntryAnalytics` / `InvoiceAnalytics`.
- **home** (Manager Home) – `contextPath: /ProjectAnalytics`; `views.paths` with `primary` = chart (`UI.PresentationVariant`) and `secondary` = table (`UI.LineItem`) → this makes it an **Analytical List Page**; `keyPerformanceIndicators`; Grid Table; outbound navigation `ProjectDetails` to `Project-manage`; libs `sap.chart`, `sap.viz`, `sap.ui.table`; no flexible column layout (one page only).
- **settings**, **auditlog** – copies of the clients app with another entity.

### 10.3 `Component.js`

```
sap.ui.define(["sap/fe/core/AppComponent"], function (Component) {   // Fiori Elements base
class
    "use strict";
    return Component.extend("psa.projects.Component", {              // name = app id +
".Component"
        metadata: { manifest: "json" }                                // everything is
configured in manifest.json
    });
});
```

### 10.4 `index.html` (to open one app alone)

```
<style> html, body, body > div, #container, #container-uiarea { height: 100%; } </style>
                                                    <!-- without it the app is blank (0 px
high) -->
<script id="sap-ui-bootstrap"
    src="https://sapui5.hana.ondemand.com/1.153.0/resources/sap-ui-core.js"   <!-- SAPUI5
from SAP CDN -->
    data-sap-ui-theme="sap_horizon"
    data-sap-ui-resource-roots='{ "psa.projects": "./" }'                     <!-- app id ->
this folder -->
    data-sap-ui-on-init="module:sap/ui/core/ComponentSupport"                 <!-- start the
component below -->
    data-sap-ui-async="true" …></script>
<body class="sapUiBody sapUiSizeCompact" id="content">
    <div data-sap-ui-component data-name="psa.projects" data-id="container" …></div>
```

---

## 11. `app/index.html` – the Fiori launchpad

1. **Who is logged in** – synchronous `GET /odata/v4/psa/whoAmI()` before the launchpad starts (fallback: normal user).
2. **Apps per role** – helper `app(intent, component, title, description)` fills `applications`: `Timesheet-manage` for everyone; for Admin also `Manager-home`, `Approval-manage`, `Project-manage`, `Invoice-manage`, `Client-manage`, `Employee-manage`, `Settings-manage`, `AuditLog-display`. Each entry: `additionalInformation: "SAPUI5.Component=psa.<app>"`, `applicationType: "URL"`, `url: "./<app>/webapp"`. Every app has its own semantic object (Fiori shows a navigation button only if exactly one app matches).
3. **`window["sap-ushell-config"]`** – the real user in the shell header (`services.Container.adapter.config`); `renderers.fiori2.componentData.config.rootIntent` = start page (Admin → `Manager-home`, User → `Timesheet-manage`); search, notifications and personalization off.
4. **Bootstrap** – `test-resources/sap/ushell/bootstrap/sandbox.js` and `sap-ui-core.js` from the SAPUI5 CDN.
5. **`attachInit`** – keep only the generated tile group (removes SAP's "Sample Applications"), `registerLogout` so *Sign Out* calls the approuter's `/do/logout`, then `createRenderer("fiori2")`.

---

# 12. Security

## 12.1 `xs-security.json` – XSUAA roles

```
"xsappname": "psa-application",                 // app name in XSUAA ($XSAPPNAME)
"tenant-mode": "dedicated",                     // single tenant
"scopes": [ { "name": "$XSAPPNAME.Admin" }, { "name": "$XSAPPNAME.User" } ],   // = roles
checked by @restrict
"role-templates":   [ { "name": "Admin", "scope-references": ["$XSAPPNAME.Admin"] }, … User
],
"role-collections": [ { "name": "PSA_Admin", "role-template-references": ["$XSAPPNAME.Admin"]
}, … PSA_User ],
                                                // assign these in the BTP cockpit to users
"oauth2-configuration": { "redirect-uris": [ "https://*.applicationstudio.cloud.sap/**",
                                             "https://*.hana.ondemand.com/**",
"http://localhost:5000/**" ] }
                                                // where XSUAA may send the user back after
login
"authorities": ["$ACCEPT_GRANTED_AUTHORITIES"], // accept authorities granted by other apps
(sample-project setting)
"oauth2-configuration": { "token-validity": 9000, … }   // token lifetime in seconds
```

Create the instance for hybrid tests: `cf create-service xsuaa application psa-hybrid-auth -c xs-security.json`.

## 12.2 Approuter – `app/`

Like the sample project, the approuter lives directly in `app/` and serves the UI files itself.

`app/package.json`: name `app`, dependency `@sap/approuter`, `start` = `node node_modules/@sap/approuter/approuter.js`.

`app/xs-app.json`:

```
"welcomeFile": "/index.html",                   // opening the URL shows the launchpad
"authenticationMethod": "route",
"sessionTimeout": 100,                          // minutes
"logout": { "logoutEndpoint": "/do/logout", "logoutPage": "/" },
"routes": [
  { "source": "^/index.html$", "target": "index.html", "localDir": "./",
    "authenticationType": "xsuaa" },            // the launchpad, from this folder
  { "source": "^/projects/webapp/(.*)$", "target": "$1", "localDir": "projects/webapp/",
    "authenticationType": "xsuaa" },            // one route per app: home, projects, …
auditlog
  …
  { "source": "^/(.*)$", "target": "$1", "destination": "srv-api",   // everything else
(/odata/…)
    "authenticationType": "xsuaa", "csrfProtection": true } ]       // goes to the CAP
server
```

`app/default-env.json` (local/hybrid only): destination `srv-api` → `http://localhost:4004`, `forwardAuthToken: true`. Locally `cds watch` serves the same `app/` files, so the URLs are the same with and without the approuter.

## 13. `mta.yaml` – deployment to Cloud Foundry

```
_schema-version: 3.3.0
ID: psa-application                # name of the MTA
version: 1.0.0
modules:
  - name: psa-application-ui       # the approuter (app/) = public URL of the app
    type: nodejs
    path: app
    requires:
      - name: srv-api
        group: destinations        # becomes destination "srv-api" (used in xs-app.json)
        properties: { name: srv-api, url: ~{srv-url}, forwardAuthToken: true, strictSSL: true
}
      - name: psa-application-auth
    provides: [ { name: app-api, properties: { app-uri: ${default-uri} } } ]
    parameters: { memory: 256M, buildpack: nodejs_buildpack }
    build-parameters: { builder: npm-ci, ignore: ["node_modules/", "default-env.json"] }
  - name: psa-application-srv      # the CAP server
    type: nodejs
    path: gen/srv
    requires: [ psa-application-auth, psa-application-db ]                    # bound XSUAA
+ HANA
    provides: [ { name: srv-api, properties: { srv-url: ${default-url} } } ]   # its URL for
the approuter
    parameters: { memory: 512M, buildpack: nodejs_buildpack }
  - name: psa-application-db-deployer   # one-off task: creates tables + views and loads the
CSVs into HANA
    type: hdb
    path: gen/db
    requires: [ psa-application-db ]
resources:
  - name: psa-application-auth     # XSUAA instance created from xs-security.json
    type: org.cloudfoundry.managed-service
    parameters:
      service: xsuaa
      service-plan: application
      path: ./xs-security.json
      config:
        xsappname: psa-application-${org}-${space}    # unique per org/space
        role-collections: [ 'PSA_Admin (${org}-${space})', 'PSA_User (${org}-${space})' ]
        oauth2-configuration: { redirect-uris: [ https://*~{app-api/app-uri}/** ] }
  - name: psa-application-db       # HDI container in SAP HANA Cloud
    type: com.sap.xs.hdi-container
    parameters: { service: hana, service-plan: hdi-shared }
build-parameters:
  before-all:                      # runs once before the modules are built
    - builder: custom
      commands:
        - npm ci                   # install exactly package-lock.json
        - npx cds build --production   # gen/srv (server) and gen/db (HANA artifacts)
```

## 14. `.pipeline/config.yml` – SAP CI/CD

Pipeline *SAP Cloud Application Programming Model*: `buildTool: mta` (mbt build), `npmExecuteScripts: true` with `runScripts: [test]` (unit tests), `cloudFoundryDeploy` (release) with API endpoint, org, space

and the credential `cf-credentials`. Used only when the CI/CD job reads its configuration from the repository.

---

## 15. `test/psa.test.js` – automated tests

```
const cds = require('@sap/cds');
const { describe, it } = require('node:test');       // Node's built-in test runner
const assert = require('node:assert/strict');
const { GET, POST, PATCH, DELETE } = cds.test(__dirname + '/..');   // starts the CAP server
in-process
const ADMIN = { auth: { username: 'meera.iyer@psa-demo.com',   password: 'admin' } };   //
mocked users
const USER  = { auth: { username: 'rahul.sharma@psa-demo.com', password: 'user'  } };
async function fails(request, status) { … }         // expects an HTTP error code
const active = (entity, key) => `${S}/${entity}(NODE_KEY=${key},IsActiveEntity=true)`;   //
saved row
const draft  = (entity, key) => `${S}/${entity}(NODE_KEY=${key},IsActiveEntity=false)`;  //
draft row
```

How a Fiori draft is simulated: `POST /ProjectSrv {}` (new draft) → `PATCH draft(...)` (type values) → `POST draft(...)/PSAService.draftActivate` (Save). Edit an existing row: `POST active(...)/PSAService.draftEdit`.

The `describe` blocks follow the FRD: login and roles, KPI analytics, seed roll-up, project insight (milestone status, burn summary), project draft, FR-003 dates, timesheet draft and rules, submit/lock, approval, import hours, alerts, invoicing (T&M, Fixed, Retainer, cancel, overdue, paid), settings and audit log. Run with `npm test`.

---

## 16. Run, deploy and give roles

**Local tests:** `npm test`. **Local app:** `npm run watch` → launchpad at `http://localhost:4004/index.html` (logins from chapter 2), or `npm run watch-<app>` for one app.

**Hybrid in BAS (real HANA + XSUAA):**

```
cf login
cf create-service xsuaa application psa-hybrid-auth -c xs-security.json
cds bind auth --to psa-hybrid-auth
cds build --production && cds deploy --to hana && cds watch --profile hybrid
# second terminal, project root:
npm install --prefix app
cds bind --exec --profile hybrid -- npm start --prefix app   # open port 5000
```

**Cloud Foundry:**

```
npm run build                    # mbt build -> mta_archives/archive.mtar
npm run deploy                   # cf deploy mta_archives/archive.mtar
```

Then, in the BTP cockpit, go to Security → Role Collections. Add users to **PSA_Admin (org-space)** or **PSA_User (org-space)**, with identity provider *Default identity provider*. Each user also needs an Employee with the same e-mail.

---

## 17. Recreate-by-hand checklist

1. `cds init`, install packages, write `package.json` (chapter 2).
2. `db/common.cds` → `db/schema.cds` (master + transaction) → `db/CDSView.cds` → `db/i18n` → CSVs; check with `cds watch` (tables + data at `/odata/v4/…`).
3. `srv/service.cds` (start with projections only, add `@restrict` and calculated columns); try it with `srv/test.http`.
4. `srv/service.js`: helpers → roll-up → project validation → timesheet rules → actions → invoices → settings/audit.
5. Generate the Fiori apps (chapter 10), then write `app/<app>/annotations.cds` app by app and add each to `app/services.cds`.
6. `app/index.html` launchpad.
7. `xs-security.json`, approuter in `app/` (`package.json`, `xs-app.json`), hybrid test with real login.
8. `mta.yaml`, `npm run build`, `npm run deploy`, assign role collections.
9. `test/psa.test.js`; `.pipeline/config.yml` for CI/CD.

**Common errors and fixes** | Error | Fix | |---|---| | HANA "incorrect syntax near =" | write `case when X = 'A' then …` (searched CASE), not `case X when 'A'` | | "no UAA service found" (approuter) | start it from the project root with `cds bind --exec … --prefix app` | | Forbidden after login / token has only `openid` | assign the role collection for the identity provider you logged in with, log out and in | | App page is blank | full-height `<style>` in `index.html`, same SAPUI5 version everywhere | | KPI card empty | value must be an `Aggregation.CustomAggregate`; a currency/unit needs one too (`@Aggregation.default: #MAX`) | | Navigation button missing in Fiori | each app needs its own semantic object |